Cheatsheet de comandos · Curso MLOps/ DataOps

ANBAN

José Picón

2026

Esto es una chuleta de uso. Toda la información operativa que necesitas durante los dos días en un solo sitio. Imprime esta hoja si lo prefieres.

Stack del curso

```
# Levantar
docker compose -f docker/docker-compose.yml up -d

# Ver estado
docker compose -f docker/docker-compose.yml ps

# Logs de un servicio
docker compose -f docker/docker-compose.yml logs -f mlflow

# Parar todo
docker compose -f docker/docker-compose.yml down

# Limpiar volúmenes (cuidado, borra datos)
docker compose -f docker/docker-compose.yml down -v
```

URLs y credenciales

Servicio	URL	Usuario / Pass
MLflow Tracking	http://localhost:5000	—
MinIO Console	http://localhost:9001	minio / minio12345
MinIO API	http://localhost:9000	minio / minio12345
Postgres	localhost:5432	mlflow / mlflow
JupyterLab	http://localhost:8888	token `anban`
Income API	http://localhost:8000	—
Swagger API	http://localhost:8000/docs	—

Variables de entorno (ponlas en cada terminal nueva)

```
export MLFLOW_TRACKING_URI=http://localhost:5000
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
export AWS_ACCESS_KEY_ID=minio
export AWS_SECRET_ACCESS_KEY=minio12345
export MODEL_URI=models:/heart-failure-clf/Staging
```

Truco: guarda esto en `setenv.sh` y haz `source setenv.sh`.

DVC

```
# Inicializar
dvc init

# Configurar remoto MinIO
dvc remote add -d minio s3://datasets/lab1
dvc remote modify minio endpointurl http://localhost:9000
dvc remote modify --local minio access_key_id minio
dvc remote modify --local minio secret_access_key minio12345

# Trackear un fichero
dvc add data/raw/heart_failure.csv

# Subir al remoto
dvc push

# Recuperar desde el remoto
dvc pull

# Ejecutar el pipeline
dvc repro

# Ver el grafo del pipeline
dvc dag

# Ver estado
dvc status

# Recuperar un fichero a su versión correcta
dvc checkout data/raw/heart_failure.csv
```

Git + DVC

```
# Commit típico tras dvc add
git add data/raw/heart_failure.csv.dvc data/raw/.gitignore
git commit -m "trackea dataset"

# Commit típico tras editar el pipeline
git add dvc.yaml dvc.lock params.yaml
git commit -m "ajusta test_size"
```

MLflow

```
# UI
open http://localhost:5000

# Listar experimentos
mlflow experiments search

# Listar runs
mlflow runs list --experiment-id 1
```

Desde Python

```
import mlflow, os
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("heart-failure-classifier")

with mlflow.start_run(run_name="rf"):
    mlflow.log_params({"n_estimators": 300})
    mlflow.log_metrics({"f1": 0.78})
    mlflow.set_tag("owner", "yo")
    mlflow.sklearn.log_model(model, name="model")
```

Model Registry

```
import mlflow
client = mlflow.MlflowClient()

# Registrar
mv = client.create_model_version(
    name="heart-failure-clf",
    source=f"runs:/{run_id}/model",
    run_id=run_id,
)

# Promocionar
client.transition_model_version_stage(
    name="heart-failure-clf",
    version=1,
    stage="Staging",
)

# Cargar
m = mlflow.pyfunc.load_model("models:/heart-failure-clf/Staging")
```

FastAPI + Docker

```
# Local con autoreload
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# Build imagen
docker build -t anban/income-api:0.1 -f Dockerfile .

# Run en Linux
docker run --rm -d --network host \
  -e MLFLOW_TRACKING_URI=http://localhost:5000 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:0.1

# Run en Mac/Windows
docker run --rm -d --network anbam_default -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:0.1

# Ver logs
docker logs -f income-api

# Stop
docker stop income-api
```

Probar la API con curl

```
# Health
curl -s http://localhost:8000/health | jq

# Version
curl -s http://localhost:8000/version | jq

# Predict (paciente con datos clínicos de insuficiencia cardíaca)
curl -s -X POST http://localhost:8000/predict \
  -H 'content-type: application/json' \
  -d '{
    "age": 75, "anaemia": 0, "creatinine_phosphokinase": 582,
    "diabetes": 0, "ejection_fraction": 20, "high_blood_pressure": 1,
    "platelets": 265000, "serum_creatinine": 1.9, "serum_sodium": 130,
    "sex": 1, "smoking": 0
  }' | jq

# Metrics
curl -s http://localhost:8000/metrics

# Monitor (drift)
curl -s http://localhost:8000/monitor | jq
```

Locust (stress test)

```
locust -f tests/locustfile.py --host http://localhost:8000 \
       --users 50 --spawn-rate 10 --run-time 30s --headless
```

Evidently y drift

```
# Generar dataset con drift
python synthetic/make_drift.py

# Reporte HTML
python src/drift_report.py \
    --reference ../lab1_dataops/data/processed/test.parquet \
    --current data/processed/drifted.parquet \
    --output reports/drift.html
```

Promote condicional

```
        python src/promote_if_better.py \
            --name heart-failure-clf \
            --candidate-version 2 \
            --metric f1 \
            --min-improvement 0.01
```

Atajos para depuración

```
# Ver puertos ocupados
lsof -i :5000

# Reiniciar un servicio
docker compose -f docker/docker-compose.yml restart minio

# Entrar a un contenedor
docker exec -it anban-mlflow bash

# Ver red de Docker
docker network ls
docker network inspect anbam_default
```

Si algo se rompe sin remedio

```
# Reset total del stack (pierdes runs)
docker compose -f docker/docker-compose.yml down -v
docker compose -f docker/docker-compose.yml up -d

# Reset del lab 1
cd labs/lab1_dataops
rm -rf data/processed
dvc repro

# Reentrenar y promocionar de cero
python labs/lab2_mlflow_dvc/src/train.py --model rf
python labs/lab2_mlflow_dvc/src/register_best.py \
    --experiment heart-failure-classifier --metric f1 --name heart-failure-clf

python labs/lab2_mlflow_dvc/src/promote.py \
    --name heart-failure-clf --version 1 --stage Staging
```